

Lab 5: IoT Weather Balloon

IoT Weather Hot-air Balloon

What is an IoT weather balloon?



Hot air balloons have been used in human history for a long time. The balloon is one of the very first things to help us understand our world from the sky. Nowadays, we have a lot of new machines, such as airplanes and drones, to help us achieve different things, but balloons are still the cheapest, the most accessible and the easiest deployment device we can use for discovering the world from the high up sky.

IoT weather balloons are one of the uses of balloons these days. These types of balloons are assembled with several devices on top of it, such as temperature sensors, pressure sensors, humidity sensors, GPS, small computing devices (RaspberryPi) and long range wireless signal antennas. There are different choices on assembling the balloon, but the most accessible one is

called pico balloon. A pico balloon is a tiny, ultra-lightweight high-altitude balloon, typically a 32-inch-diameter party balloon filled with hydrogen or helium, designed to carry a lightweight (often <50g) electronics payload, such as a Raspberry Pi Pico or a WSPR transmitter, into the stratosphere (approx. 45,000 ft). These superpressure balloons travel around the world for weeks or months, powered by solar energy and tracked via amateur radio networks.

In this lab, you are going to write programs to analyze the sensor data gathered by the pico balloons. We provide you with the sample of the sensor data and your program needs to achieve the tasks we listed below.

What will you learn in this lab?

In this lab, you will learn several things:

- A real IoT application (weather balloon), which will give perspective on general IoT application development
- IoT data dissemination protocols, why they are important, and how they are implemented
- Practical IoT application, such as location prediction

In this lab, **you will design, build, and deploy a real IoT application from the ground up**. You will take the role of a designer/operator working for a wildlife refuge in sub-saharan Africa. You are responsible for a 50 square mile territory. On your territory is a diverse range of animals, including zebras, lions, and elephants. Since this is a big area, you can't simply walk around and watch the animals. Instead, **you will perform tracking, where you affix IoT sensors to the animals, which monitor attributes about the animal such as its location, movement patterns and so on.**

Step 1: Predict Fall Down Location Based on GPS Data

In this section, you will analyze the data collected from the GPS on the balloon. We want to find the fall down location based on it. To predict the fall down location, you first need to draw a wind direction map from the GPS data to see the balloon's route. Then predict the fall down location based on the GPS data associated with the expected balloon live time. You can find the GPS data [here](#).

Step 1a: Draw wind direction map

Take a look at the program below. Fill up the blank in the program. You should let the program output the result as a figure. Put this figure in your report. The example of the wind map figure is shown after the sample code.

```

"""
Program 1: Wind Direction Map
Reads gps_data.csv and plots the balloon's route with bold arrows
showing direction of travel. Arrows are colored by speed.
"""
import csv
import math
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.colors as mcolors
import matplotlib.cm as cm

def load_gps(path):
    """
    Load GPS data from a CSV file.

    Reads the file and extracts the time_s, latitude, and longitude
    columns into separate lists.

    Args:
        path (str): Path to the GPS CSV file.

    Returns:
        times (list): List of timestamps in seconds.
        lats (list): List of latitude values.
        lons (list): List of longitude values.
    """
    times, lats, lons = [], [], []
    with open(path, newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            times.append(float(row["time_s"]))
            lats.append(float(row["latitude"]))
            lons.append(float(row["longitude"]))
    return times, lats, lons

def plot_wind_map(lats, lons, step, title, out_path):
    """
    Plot a wind direction map showing the balloon's route with arrows.

    This function should:
    1. Create a matplotlib figure and axis.
    2. Loop through the GPS data with a stride of (2 * step) to create
       non-overlapping arrow segments. For each segment from index i to
       i + step, compute:
    """

```

- du = longitude difference (lons[i+step] - lons[i])
 - dv = latitude difference (lats[i+step] - lats[i])
 - speed = magnitude of (du, dv) using math.sqrt
3. Normalize the speed values and use a colormap (e.g. "viridis") to assign a color to each arrow based on its speed.
 4. Draw each arrow using matplotlib.patches.FancyArrowPatch with the computed start point, end point, and color.
 5. Add a colorbar, axis labels, title, and grid.
 6. Save the figure to out_path.

Args:

```

lats (list):    List of latitude values from GPS data.
lons (list):    List of longitude values from GPS data.
step (int):     Number of rows between arrow start and end.
title (str):    Title for the plot.
out_path (str): File path to save the output figure.

```

"""

```

# TODO: Create a figure and axis using plt.subplots()

# TODO: Loop through the data to build arrow segments
#       Use range(0, len(lats) - step, 2 * step) so arrows
#       don't share start/end points
#       For each segment, compute du, dv, and speed

# TODO: Normalize speeds using mcolors.Normalize and get a
#       colormap using cm.get_cmap("viridis")

# TODO: Draw each arrow using mpatches.FancyArrowPatch
#       Set arrowstyle, linewidth, color from colormap

# TODO: Add a colorbar using cm.ScalarMappable

# TODO: Set axis labels, title, aspect ratio, and grid

# TODO: Save the figure using plt.savefig()

```

pass

def main():

"""

Main function that loads GPS data and generates the wind map.

"""

```

# Load GPS data from the provided CSV file
times, lats, lons = load_gps("gps_data.csv")

```

```

# Generate wind direction map

```

```

plot_wind_map(
    lats, lons,
    step=10,

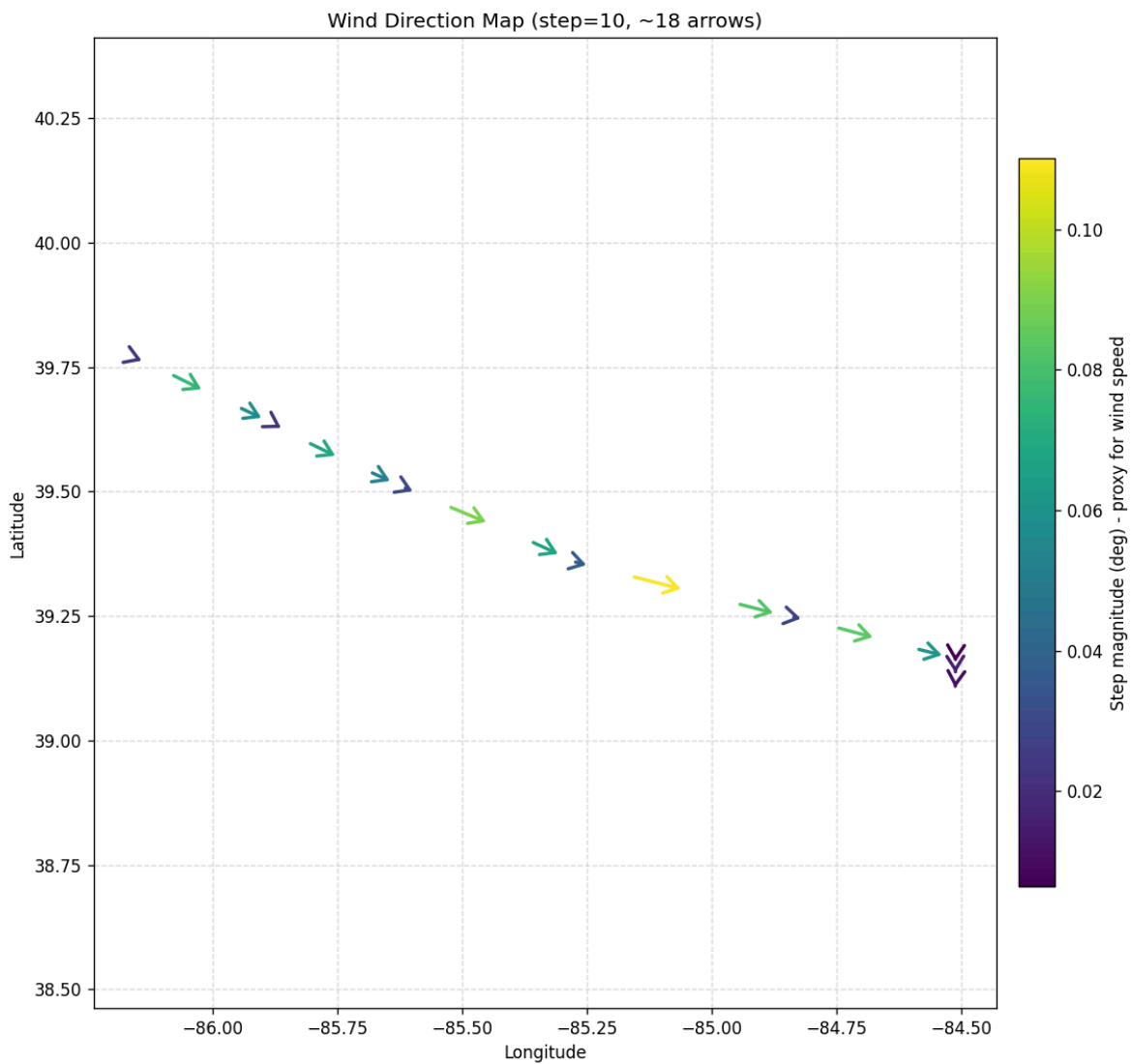
```

```

    title="Wind Direction Map",
    out_path="wind_map.png",
)

if __name__ == "__main__":
    main()

```



Step 1b: Predict the fall down location

Our balloon is expected to run for 3 hours. Based on this info, try your best to predict the fall down location for it. You can use the program skeleton here. Put the coordinate (latitude, longitude) of the fall down location and give the detail on how you predict it in your report.

Extra point: listing at least four different models that can generate the predicted location.
Answer the questions: Which model do you think will do the best prediction? Why?

```
"""
Program 2: Fall-down Location Predictor
Given the time the balloon runs out of helium, predict where it will
fall.

Approach: fit a degree-2 polynomial to lat(t) and lon(t) using the
available GPS data, then evaluate at runout_time. Assumes the balloon
drops straight down when helium runs out (altitude is ignored).
"""
import csv

import numpy as np

def load_gps(path):
    """
    Load GPS data from a CSV file.

    Reads the file and extracts the time_s, latitude, and longitude
    columns into numpy arrays for numerical computation.

    Args:
        path (str): Path to the GPS CSV file.

    Returns:
        times (np.array): Array of timestamps in seconds.
        lats (np.array): Array of latitude values.
        lons (np.array): Array of longitude values.
    """
    times, lats, lons = [], [], []
    with open(path, newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            times.append(float(row["time_s"]))
            lats.append(float(row["latitude"]))
            lons.append(float(row["longitude"]))
    return np.array(times), np.array(lats), np.array(lons)

def predict_fall_location(gps_path, runout_time_s):
    """
    Predict the fall-down location of the balloon.

    This function should:
    1. Load the GPS data using load_gps().
    2. Use np.polyfit() to fit a degree-2 polynomial to latitude
    """
```

```

    as a function of time.
3. Use np.polyfit() to fit a degree-2 polynomial to longitude
   as a function of time.
4. Use np.polyval() to evaluate both polynomials at runout_time_s
   to get the predicted latitude and longitude.

Args:
    gps_path (str): Path to the GPS data file.
    runout_time_s (float): The time (in seconds) when helium runs
out.

Returns:
    tuple: (predicted_latitude, predicted_longitude)
"""
# TODO: Load GPS data
times, lats, lons = ____

# TODO: Fit a degree-2 polynomial for latitude over time
lat_coeffs = ____

# TODO: Fit a degree-2 polynomial for longitude over time
lon_coeffs = ____

# TODO: Evaluate the latitude polynomial at runout_time_s
pred_lat = ____

# TODO: Evaluate the longitude polynomial at runout_time_s
pred_lon = ____

return float(pred_lat), float(pred_lon)

def main():
    """
    Main function that predicts the balloon's fall-down location.

    The balloon is expected to run for 3 hours (10800 seconds).
    """
    # Path to the GPS data file
    gps_path = "gps_data.csv"

    # Balloon runout time in seconds (3 hours)
    runout_time = 10800

    # Predict the fall-down location
    lat, lon = predict_fall_location(gps_path, runout_time)

    # Print the predicted coordinates
    print(f"Predicted fall-down location at t={runout_time}s:")
    print(f"    latitude = {lat:.6f}")

```

```
print(f" longitude = {lon:.6f}")

if __name__ == "__main__":
    main()
```

Step 2: Send Data to Radio Tower

A PICO balloon usually uses two types of the wireless protocol, Automatic Packet Reporting System (APRS) and Weak Signal Propagation Reporter (WSPR). In this step, you are going to practice how to encode the fall down location coordinate to a WSPR package. This package will be sent to the radio tower so we can go to the location beforehand to collect the balloon.

WSPR is an open source protocol which is designed for probing potential propagation paths with low-power transmissions. It is usually used for message exchange in between two radio towers. The standard WSPR message contents <callsign> + <4 character locator> + <dBm transmit power>. A call sign is a unique identifier for a transmitter station, operator, or vehicle used in radio communications, broadcasting, and aviation. The locator indicates the sender's location. For example, the locator of the radio tower in Chicago is EN61. The transmit power is the power level we use to send the message. The range of the power level should be between 0 to 60 dBm. Given the format above, one possible WSPR message can be [WB1UI EN61 30].

To encode the WSPR message, there are rules to be followed. The detailed coding process can be found in [this](#) document. For callsign, there are six characters. Each character only contains 0-9 (coded by 0-9), A-Z (coded by 10-35) and space (coded by 36), 37 possibilities in total. However, only the first character has all 37 choices. The second character cannot be space, so only 36 possibilities. The third character can only be numbers, which only has 10 possibilities. The fourth to sixth character cannot be numbers, which only have 27 possibilities (which need to be minus 10 when coded). Thus, given the example above, WB1UI can be coded into [32 11 1 20 8 26]. Now, to transfer it into a hexadecimal number by this formula:

$$((((([Ch1]*36+[Ch2])*10+[Ch3])*27+[Ch4])*27+[Ch5])*27+[Ch6])$$

For the above example WB1UI, the number is DA57753.

For the locator, it is usually pre-defined by the location of each radio tower. The details of how to encode the locator is described in the document linked above. For this lab, we want you to encode the predicted fall down location, rather than the radio tower locator. Given that WSPR only has 15 bits for the locator, we will first make the latitude and longitude into a small float number by minus the latitude and longitude of the start point of the balloon (the first row in the GPS dataset). For example, the start point is [4.1, 3.1], and the predicted fall down location is [4.5, 4.2], then the calculated coordinate should be [0.4, 1.1]. Note: always take the absolute

value of each number in coordinate. Now, transfer the float point into a 8 bit float. The way that we calculate the 8 bit float is by changing the bit of exponent and mantissa in the floating point format. An 8 bit float usually goes with 3 bits for exponent and 4 bits for mantissa. In Python, there is a package called **bitstring**. Here is an example code for it:

```
>>> from bitstring import Bits
>>> Bits(e4m3float=0.4).pp
<bound method Bits.pp of Bits('0x34')>
```

You can learn more about this package [here](#). We only take the 7 bits on the right side and get rid of the very left bit since we only consider the float to be positive. After the transformation, we can concatenate the 7bit of latitude and 7 bit of longitude with a 0 on the very left side to make the final number become 15 bits. Given the example [0.4, 1.1], the 15 bits should be 001101001000000.

Finally, we can concatenate the power level with the coordinate. The power level in the WSPR is coded into 7 bits. In this lab, we set the power level to be 30 dBm, which is encoded into 0011110. Attach these bits to your 15 bits coordinate and transfer it into a hexadecimal number as the result. For example, using the 15 bits coordinate created above, the 22 bits are 0011010010000000011110, which is 0D201E in hexadecimal.

In the report, you need to set your own callsign, and the transferred hexadecimal number of it. Also, the coordinate of your predicted location combined with the 30 dBm power level. In short, your report should have

1. Your own callsign
2. The hexadecimal number of your call sign
3. The predicted fall down location from previous step
4. The hexadecimal number of the location concatenated with 30 dBm power level

Bonus: Analyze Sensor Data

Our balloon not only installed a GPS sensor, but also other sensors. [Here](#) are the other sensor data, including temperature, pressure and humidity. Please do some analysis on those data on your own preference. Put your results in the report and answer the following questions.

1. What do you find based on the data?
2. How do you analyze it to get the previous conclusion?
3. Is there another analyzing process/algorithm/method that can improve your results?
4. What other sensors can be added to the balloon?

Submission and Grading

The rubric for grading that you should follow for this lab is provided here. Each team should submit a single report with the net-ids of the team members listed in it.

1. **Step 1a: Did the student correctly create the wind map based on the given GPS data?**
 - a. If the students write a python program to create the wind map, they will get 5 points.
 - b. If the wind map created is correct on the wind direction, the students get 10 points.
 - c. If the wind map has colors indicating the wind speed, the students get 5 points.
 - d. If the wind map is hard to read, the student should be deducted 5 points.
2. **Step 1b: Did the student write the program to predict the fall down location?**
 - a. If the students write a python program to predict the location, they get 5 points.
 - b. If the students put the predicted location coordinates in the report, they get 5 points.
3. **Step 1b extra credits: Did the student discuss the questions?** Question: listing at least four different models that can generate the predicted location. Which model do you think will do the best prediction? Why? 5 points.
4. **Step 2: Did the student correctly transfer the predicted location into the WSPR message?**
 - a. If the students give the callsign and correctly transfer it into a hexadecimal number based on WSPR rule, they get 5 points.
 - b. If the students transfer the location coordinate and power level (30 dBm) into a hexadecimal number based on WSPR rule, they get 5 points.
5. **Bonus points: Did the student answer the 4 questions related to this lab?**
 - a. Each of the questions is worth for 5 points.

Submission	Points
Step 1a	20
Step 1b	10
Step 1b Extra Credits	5
Step 2	10
Bonus Points	20

Total/Total with Bonus	40/65
-------------------------------	--------------

Note: Please add a table to your report listing all team members and their respective contributions.